

Manila Rain Frequency Dashboard

AI Bootcamp Final Project — Predictive Model + AI Assistant

Prepared by: Javier, Jay-Ron
R.

Accenture AI Bootcamp
(ATCP Resources)

June 2026

Live App: <https://manila-rain-dashboard.vercel.app/>

Component	Points
Data Loading and Exploration	5
Data Cleaning and Pre-processing	10
Data Visualization	5
Prediction Model	15
Azure OpenAI Service Setup	20
Azure Cognitive Services Implementation	20
Prompt Engineering	15
Create & Integrate AI to Web App	15
Presentation	20
Total	130

Table of Contents

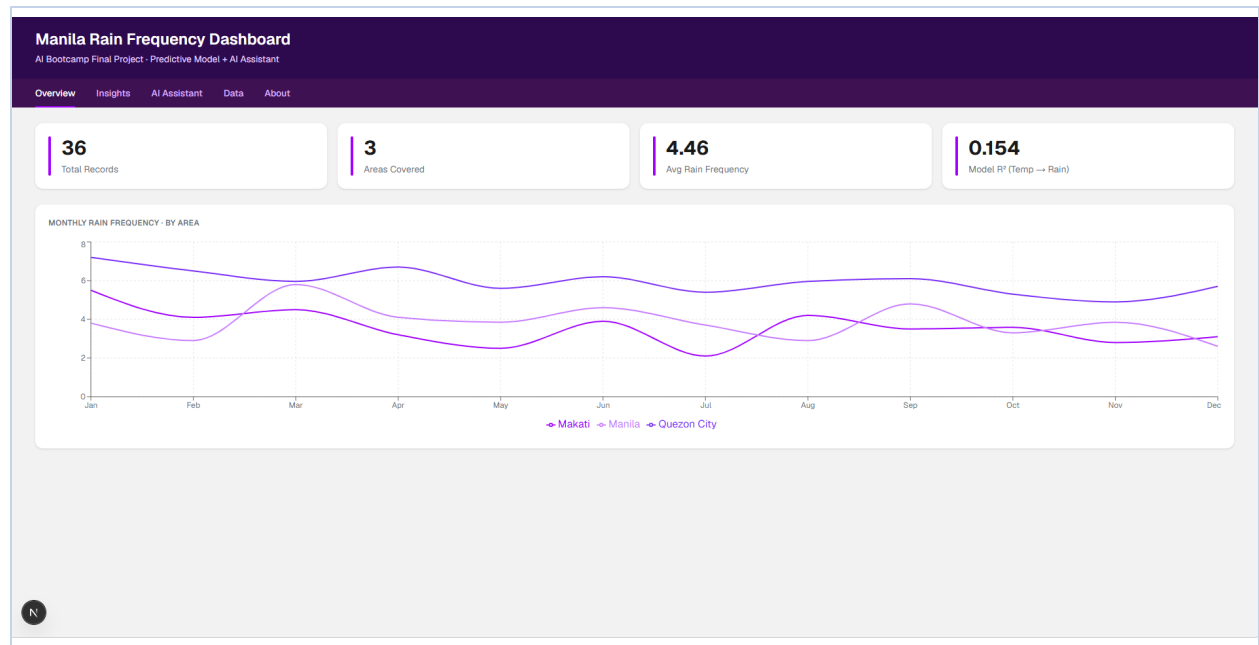
- 1 · Project Overview
- 2 · Dataset
- 3 · Data Cleaning
- 4 · Data Visualizations
- 5 · Prediction Model
- 6 · Azure OpenAI Integration
- 7 · Azure Speech (Text-to-Speech)
- 8 · Prompt Engineering
- 9 · Web App — Features & Architecture
- 10 · Q&A; Preparation

Section 1 · Project Overview

Rain frequency in Metro Manila varies significantly across areas and months. This project builds a machine learning model to predict rain frequency from temperature data, and integrates the model into a fully deployed AI-powered web dashboard with a conversational assistant and text-to-speech capabilities.

- Problem** Dataset contains weather parameters for Manila areas with missing values and inconsistencies. Goal: clean the data, build a predictive model, and expose insights via an AI chatbot web app.
- Solution** Azure Databricks for data processing and model training. Azure OpenAI for the chatbot. Azure Speech for text-to-speech. Next.js + Vercel for the deployed web app.
- Beyond Minimum** Built a multi-dataset selector with Azure Blob Storage manifest pattern — users can upload multiple datasets and switch between them live without redeploying.

Layer	Technology
Data Processing & ML	Azure Databricks (Python · Pandas · Scikit-Learn)
LLM	Azure OpenAI — gpt-4.1-mini (Responses API)
Text-to-Speech	Azure Speech Service — Jenny Neural voice
Frontend	Next.js App Router · Tailwind CSS · Recharts
Backend API	Next.js API Routes (server-side, keys never reach browser)
Storage	Azure Blob Storage (dataset blobs + manifest.json)
Deployment	Vercel + Personal GitHub



Web App — Overview tab: KPI cards (36 records, 3 areas, avg rain frequency 4.46, Model R^2 0.154) and monthly rain frequency trend by area.

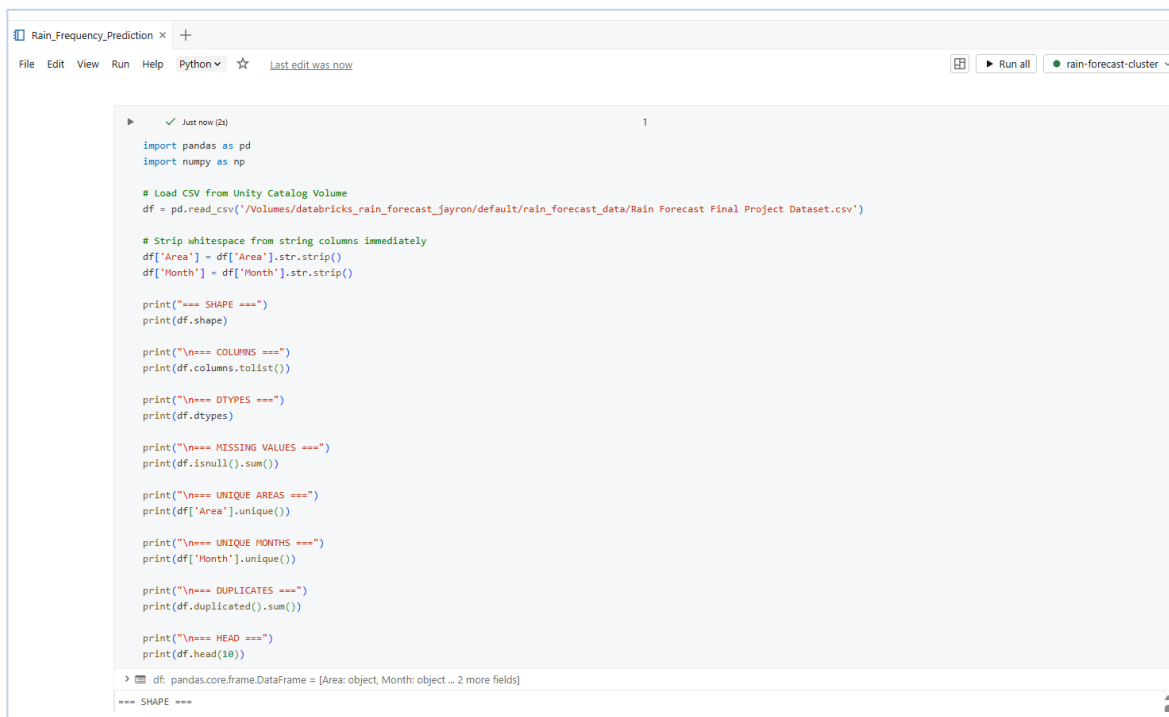
Section 2 · Dataset

Rubric: Data Loading and Exploration — 5 pts

The dataset contains 36 rows across 3 Manila areas (Makati, Quezon City, Manila) and 12 months, with 4 columns: Area, Month, Temperature, and Rain Frequency. It had missing values in both numeric columns and whitespace issues in string columns.

Column	Type	Issue Found
Area	String	Leading/trailing whitespace — fixed with .strip()
Month	String	Leading/trailing whitespace — fixed with .strip()
Temperature	Float	1 missing value (Quezon City, June)
Rain Frequency	Float	5 missing values across all 3 areas

Important: Dataset shape: 36 rows × 4 columns. Areas: Makati, Quezon City, Manila. Months: January–December. Duplicates: 0.



```
import pandas as pd
import numpy as np

# Load CSV from Unity Catalog Volume
df = pd.read_csv('/Volumes/databricks_rain_forecast_jayron/default/rain_forecast_data/Rain Forecast Final Project Dataset.csv')

# Strip whitespace from string columns immediately
df['Area'] = df['Area'].str.strip()
df['Month'] = df['Month'].str.strip()

print("=== SHAPE ===")
print(df.shape)

print("\n=== COLUMNS ===")
print(df.columns.tolist())

print("\n=== DTYPES ===")
print(df.dtypes)

print("\n=== MISSING VALUES ===")
print(df.isnull().sum())

print("\n=== UNIQUE AREAS ===")
print(df['Area'].unique())

print("\n=== UNIQUE MONTHS ===")
print(df['Month'].unique())

print("\n=== DUPLICATES ===")
print(df.duplicated().sum())

print("\n=== HEAD ===")
print(df.head(10))
```

> df: pandas.core.frame.DataFrame = [Area: object, Month: object ... 2 more fields]

=== SHAPE ===

Databricks Notebook — Cell 1 code: loading CSV from Unity Catalog Volume, stripping whitespace.

```
1  ==> Unnamed: 0: pandas.core.frame.DataFrame = [Area: object, Month: object, ... 2 more items]

=== SHAPE ===
(36, 4)

=== COLUMNS ===
['Area', 'Month', 'Temperature', 'Rain Frequency']

=== DYPES ===
Area          object
Month         object
Temperature   float64
Rain Frequency float64
dtype: object

=== MISSING VALUES ===
Area          0
Month         0
Temperature    1
Rain Frequency 5
dtype: int64

=== UNIQUE AREAS ===
```

Databricks Notebook — Cell 1 output: shape (36, 4), columns, dtypes, missing value counts (Temperature: 1, Rain Frequency: 5).

Section 3 · Data Cleaning

Rubric: Data Cleaning and Pre-processing — 10 pts

Missing values were filled using the group mean per area — not a global mean — to preserve each area's distinct rainfall pattern. Makati, Quezon City, and Manila have different rainfall baselines, so using a global mean would have distorted the data.

- **Strategy** Group mean per Area for both Temperature and Rain Frequency. This is more accurate than a global mean because QC consistently has higher rain frequency than Makati/Manila at similar temperatures.
- **Month Encoding** Month names converted to numeric order (Jan=1 ... Dec=12) for ML compatibility.
- **Area Encoding** Area names encoded as integers using LabelEncoder for model training.

Important: After cleaning, zero nulls remained across all columns. The cleaned dataset was exported as dataset_context.json and uploaded to Azure Blob Storage for use by the web app.

```
Just now (<1s) 2

# Fill missing Temperature and Rain Frequency using the mean of that Area
df['Temperature'] = df.groupby('Area')['Temperature'].transform(lambda x: x.fillna(x.mean()))
df['Rain Frequency'] = df.groupby('Area')['Rain Frequency'].transform(lambda x: x.fillna(x.mean()))

print("=== MISSING VALUES AFTER CLEANING ===")
print(df.isnull().sum())

print("\n=== SHAPE AFTER CLEANING ===")
print(df.shape)

print("\n=== SUMMARY STATS PER AREA ===")
print(df.groupby('Area')[['Temperature', 'Rain Frequency']].mean())

print("\n=== FULL CLEANED DATA ===")
print(df.to_string())

=== MISSING VALUES AFTER CLEANING ===
Area      0
Month      0
Temperature  0
Rain Frequency  0
dtype: int64

=== SHAPE AFTER CLEANING ===
(36, 4)

=== SUMMARY STATS PER AREA ===
              Temperature  Rain Frequency
Area
Makati      23.416667      3.581818
Manila      27.250000      3.850000
Quezon City  25.454545      5.960000

=== FULL CLEANED DATA ===
   Area  Month  Temperature  Rain Frequency
0  Makati  January    28.000000      5.500000
1  Quezon City  January    29.000000      7.200000
```

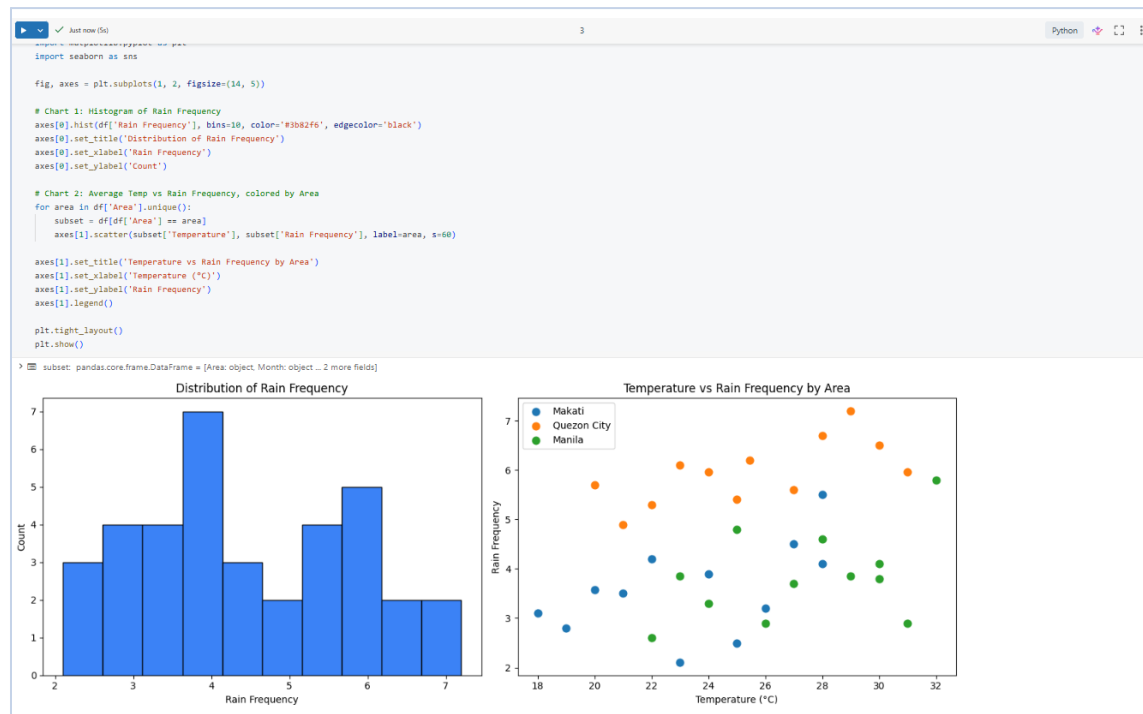
Databricks Notebook — Cell 2 output: zero nulls confirmed after cleaning, shape still (36, 4), per-area summary stats, and full cleaned dataset preview.

Section 4 · Data Visualizations

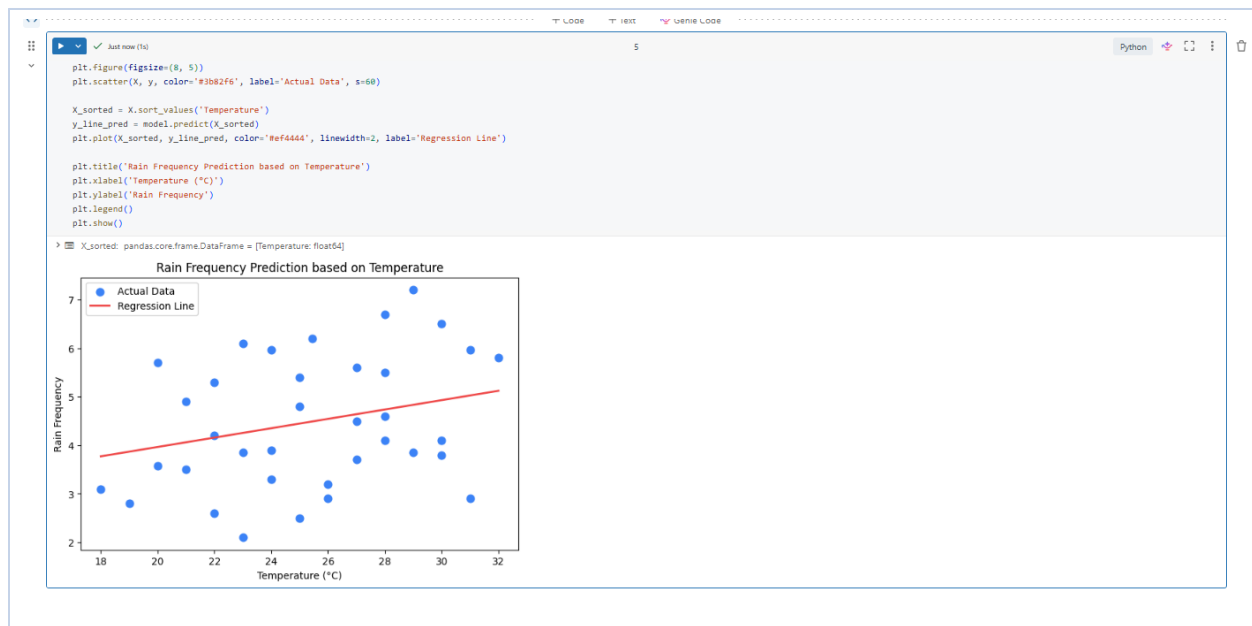
Rubric: Data Visualization — 5 pts

Charts were generated in Databricks to explore the dataset before model training, and the same relationships are visualized live in the deployed web app's Overview and Insights tabs.

- **Distribution & Area Scatter** Histogram of Rain Frequency across all records, alongside a Temperature vs. Rain Frequency scatter colored by area — both generated in Databricks.
- **Monthly Trend (Web App)** Line chart of Rain Frequency by Month per Area, rendered live in the Overview tab from the active dataset in Azure Blob Storage.
- **Predictive Chart** Scatter plot with regression line overlay showing the model's predicted Rain Frequency vs. actual values, generated both in Databricks and in the web app's Insights tab.



Databricks Notebook — Cell 3 output: Distribution of Rain Frequency histogram (left) and Temperature vs. Rain Frequency by Area scatter (right).



Databricks Notebook — Cell 5 output: Rain Frequency Prediction based on Temperature — actual data points with regression line overlay.



Web App — Insights tab: Temperature vs. Rain Frequency scatter (top) and Predictive Model linear regression chart (bottom) with RMSE/R² caption.

Section 5 · Prediction Model

Rubric: Prediction Model — 15 pts

A Linear Regression model was trained using Temperature as the sole predictor of Rain Frequency, per assignment spec. A second model adding Area as a feature was also tested as a comparison.

Model	Features	RMSE	R ²	Notes
Model 1 (Primary)	Temperature only	1.2263	0.1540	Spec-required deliverable
Model 2 (Comparison)	Temperature + Area	0.8064	0.6341	Area is stronger driver

Important: Why is R^2 low? With only 36 rows and Temperature as the sole feature, $R^2 = 0.154$ means Temperature explains roughly 15% of Rain Frequency variation. Adding Area raises R^2 to 0.634 — confirming location is the stronger predictor. Model 1 is kept as the primary deliverable since it matches the assignment spec exactly; Model 2 demonstrates deeper analysis. The regression formula (Rain Frequency = intercept + coefficient × Temperature) is embedded in the web app and used live by the AI assistant to answer prediction questions.

+

Code

+

Text

+

Genie Code

Python

+

+

+

+

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

X = df[['Temperature']]
y = df['Rain Frequency']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)

print("=== MODEL COEFFICIENTS ===")
print(f"Intercept: {model.intercept_:.4f}")
print(f"Coefficient (Temperature): {model.coef_[0]:.4f}")

print("\n=== MODEL PERFORMANCE ===")
print(f"RMSE: {rmse:.4f}")
print(f"R^2 score: {r2:.4f}")

print("\n=== TEST SET PREDICTIONS vs ACTUAL ===")
results = pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred})
print(results)
```

(1) MFlow run

Logged 1 run to an experiment in MFlow. Learn more

>

results: pandas.core.frame.DataFrame = [Actual: float64, Predicted: float64]

>

X: pandas.core.frame.DataFrame = [Temperature: float64]

>

X_test: pandas.core.frame.DataFrame = [Temperature: float64]

>

X_train: pandas.core.frame.DataFrame = [Temperature: float64]

=== MODEL COEFFICIENTS ===

Intercept: 2.8347

Coefficient (Temperature): 0.0000

Databricks Notebook — Cell 4 code: Linear Regression Model 1 (Temperature only) training with sklearn.

```
> results: pandas.core.frame.DataFrame = [Actual: float64, Predicted: float64]
> X: pandas.core.frame.DataFrame = [Temperature: float64]
> X_test: pandas.core.frame.DataFrame = [Temperature: float64]
> X_train: pandas.core.frame.DataFrame = [Temperature: float64]

=== MODEL COEFFICIENTS ===
Intercept: 2.0347
Coefficient (Temperature): 0.0965

=== MODEL PERFORMANCE ===
RMSE: 1.2263
R² Score: 0.1548

=== TEST SET PREDICTIONS vs ACTUAL ===
Actual Predicted
0 2.6 4.156937
1 5.6 4.639261
2 4.8 4.446331
3 2.8 3.867542
4 6.2 4.498179
5 4.9 4.868872
6 4.2 4.156937
7 2.5 4.446331
```

Tip: Use display() with this Pandas DataFrame to access interactive visualization features. Don't show again

Display dataframe

Databricks Autologging created an MLflow Run: Hyperparameters, performance metrics, model artifacts, and lineage information for the current workflow were automatically tracked with MLflow. [Learn more](#)

Try machine learning in Genie Code: Genie Code can help you build, debug, and iterate on machine learning code. [Open Genie Code](#) to get started.

Databricks Notebook — Cell 4 output: Model 1 coefficients (Intercept: 2.0347, Coefficient: 0.0965), RMSE: 1.2263, R^2 Score: 0.1540, test set predictions vs actual.

```
df_encoded = pd.get_dummies(df, columns=['Area'], drop_first=True)

feature_cols = ['Temperature'] + [c for c in df_encoded.columns if c.startswith('Area_')]
X2 = df_encoded[feature_cols]
y2 = df_encoded['Rain Frequency']

X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2, random_state=42)

model2 = LinearRegression()
model2.fit(X2_train, y2_train)
y2_pred = model2.predict(X2_test)

rmse2 = mean_squared_error(y2_test, y2_pred) ** 0.5
r2_2 = r2_score(y2_test, y2_pred)

print("=== MODEL 2 (Temperature + Area) PERFORMANCE ===")
print(f"RMSE: {rmse2:.4f}")
print(f"R² Score: {r2_2:.4f}")

print("\n=== COMPARISON ===")
print(f"Model 1 (Temperature only): RMSE={rmse:.4f}, R²={r2:.4f}")
print(f"Model 2 (Temperature + Area): RMSE={rmse2:.4f}, R²={r2_2:.4f}")
```

MLflow run
Logged 1 run to an experiment in MLflow. [Learn more](#)

```
> df_encoded: pandas.core.frame.DataFrame = [Month: object, Temperature: float64 ... 3 more fields]
> X2: pandas.core.frame.DataFrame = [Temperature: float64, Area_Manila: bool ... 1 more field]
> X2_test: pandas.core.frame.DataFrame = [Temperature: float64, Area_Manila: bool ... 1 more field]
> X2_train: pandas.core.frame.DataFrame = [Temperature: float64, Area_Manila: bool ... 1 more field]

=== MODEL 2 (Temperature + Area) PERFORMANCE ===
RMSE: 0.8064
R² Score: 0.6341

=== COMPARISON ===
Model 1 (Temperature only): RMSE=1.2263, R²=0.1548
Model 2 (Temperature + Area): RMSE=0.8064, R²=0.6341
```

Databricks Notebook — Cell 6 output: Model 2 (Temperature + Area) RMSE: 0.8064, R^2 : 0.6341, side-by-side comparison of both models.

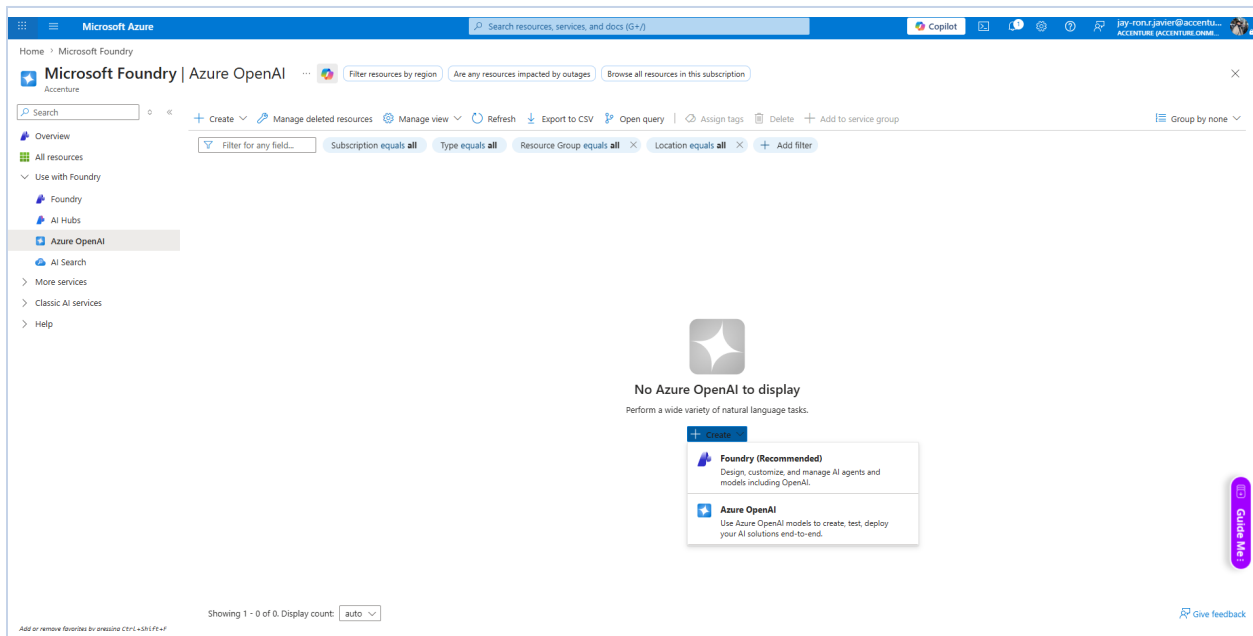
Section 6 · Azure OpenAI Integration

Rubric: Azure OpenAI Service Setup — 20 pts

Azure OpenAI powers the AI Assistant tab. The gpt-4.1-mini model is deployed via Microsoft Foundry and called using the Responses API from a secure Next.js server-side route — API keys never reach the browser.

- **Resource** openai-rain-forecast-jayron (East US). Deployment: gpt41-mini-manila-chatbot via Foundry-managed resource jay-ronrjavier-4102-resource.
- **API Shape** Uses the Responses API (/openai/v1/responses) — not the older Chat Completions format. Foundry provisions this by default as of mid-2026.
- **Security** API key stored as a Vercel environment variable. Only accessed in the /api/chat route. Never exposed to the client.

Important: Troubleshooting hit: gpt-35-turbo and gpt-4o-mini (2024-07-18) were both deprecated/unavailable. gpt-4.1-mini used instead. Also hit DeploymentNotFound — root cause was the Foundry-managed resource having a different endpoint than the originally created Azure OpenAI resource.



Microsoft Foundry | Azure OpenAI — no existing resources shown, prompting creation via Foundry (Recommended) flow.

Home > Microsoft Foundry | Azure OpenAI

Create Azure OpenAI

✓ Basics

✓ Network

✓ Tags

4 Review + submit

View automation template

Basics

SubscriptionVisual Studio Enterprise Subscription

Resource grouprg-rain-forecast-project-jayron

RegionEast US

Nameopenai-rain-forecast-jayron

Pricing tierStandard S0

Network

TypeAll networks, including the internet, can access this resource.

Create Azure OpenAI — Review + submit: resource name openai-rain-forecast-jayron, East US, Standard S0, all networks allowed.

Home > Resource Manager | Resource groups > rg-rain-forecast-project-jayron

openai-rain-forecast-jayron

Azure OpenAI

Search

Go to Foundry portal

Delete

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Resource Management

Security

Monitoring

Automation

Help

Want to try the latest industry models and Agents ?

Azure OpenAI is limited to OpenAI models. Upgrading your resource type to Foundry gives you access to Open AI plus third-party models like Meta , Mistral , uAI and more. Build enterprise grade AI agents using your data and tools.

Get Started

Essentials

Resource group (move) : rg-rain-forecast-project-jayron

Status : Active

Location : East US

Subscription (move) : Visual Studio Enterprise Subscription

Subscription ID : f09de20a-1029-4e7c-8b23-1e0580a9a48f

Tags (edit) : Add tags

API Kind : OpenAI

Pricing tier : Standard

Endpoints : [Click here to view endpoints](#)

Manage keys : [Click here to manage keys](#)

JSON View

Get Started

Develop

Monitor

Build your own secure copilot and generative AI applications with Azure OpenAI Service

Deploy an Azure OpenAI model and start making API calls. Connect your own data, call functions, and improve workflow with Azure OpenAI language, image and speech models. You can access the service through REST APIs, Python SDK, or our web-based interface in the Foundry portal.

Learn More

Explore and deploy

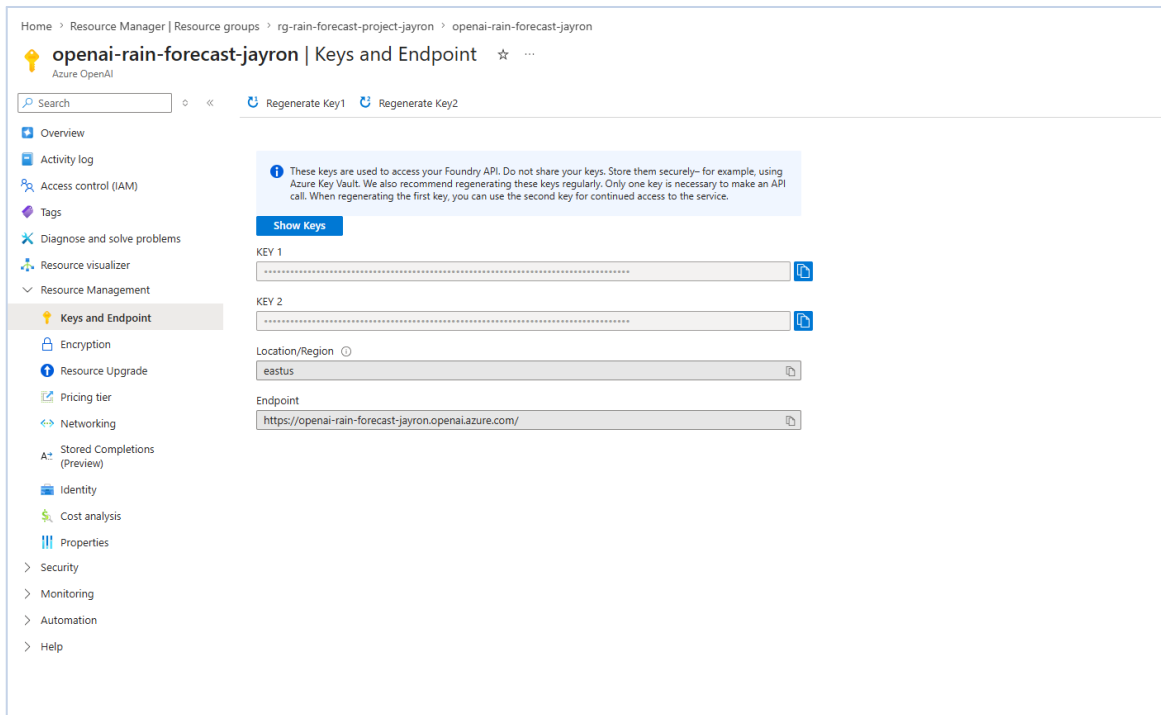
Explore and deploy the generative AI models, craft unique prompts for your use cases, and fine-tune select models.

Explore Foundry portal

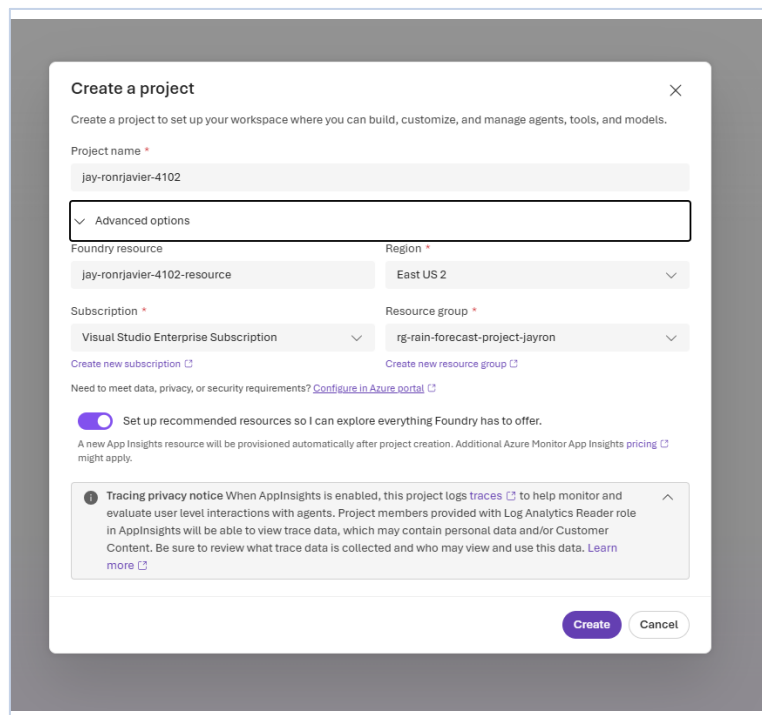
Guides Me

Add or remove favorites by pressing Ctrl+L+Shift+F

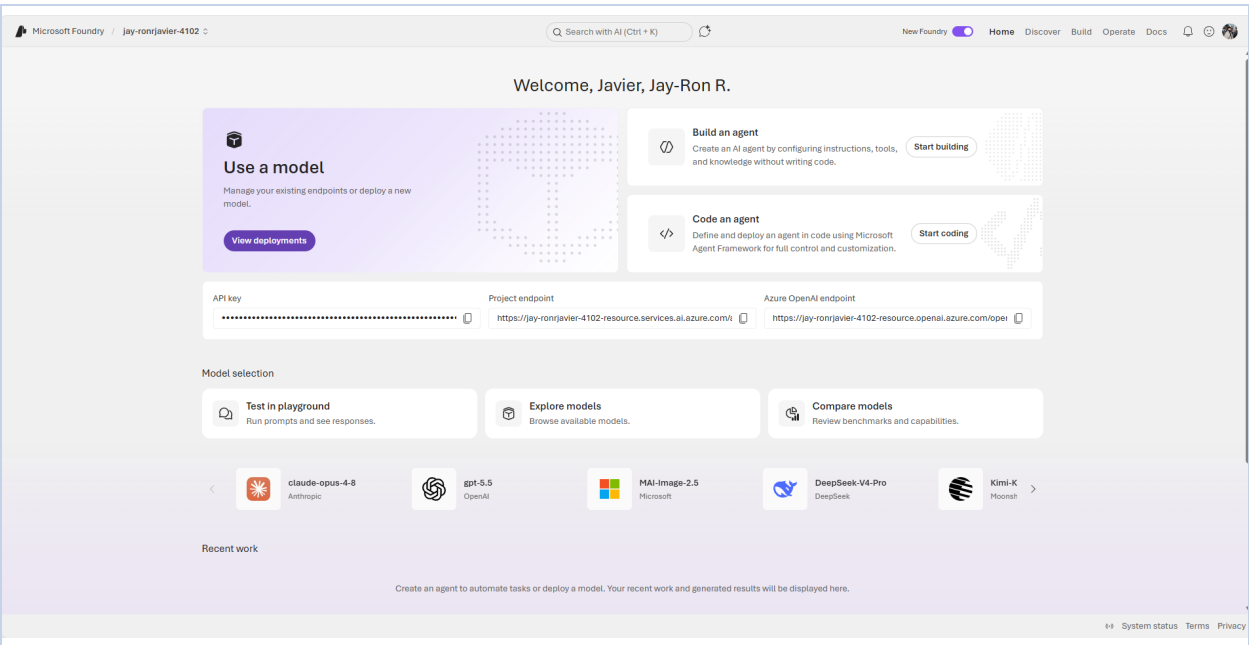
Azure Portal — openai-rain-forecast-jayron resource overview: Status Active, East US, Standard pricing tier.



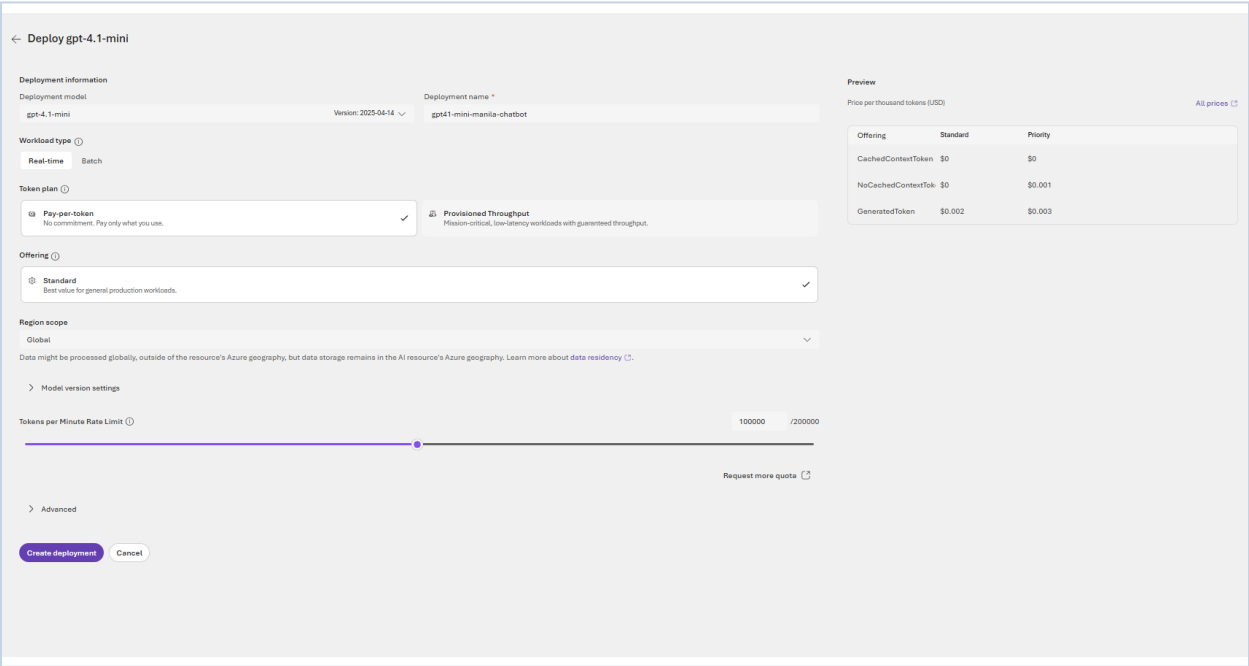
Azure Portal — openai-rain-forecast-jayron Keys and Endpoint: eastus region, endpoint <https://openai-rain-forecast-jayron.openai.azure.com/>



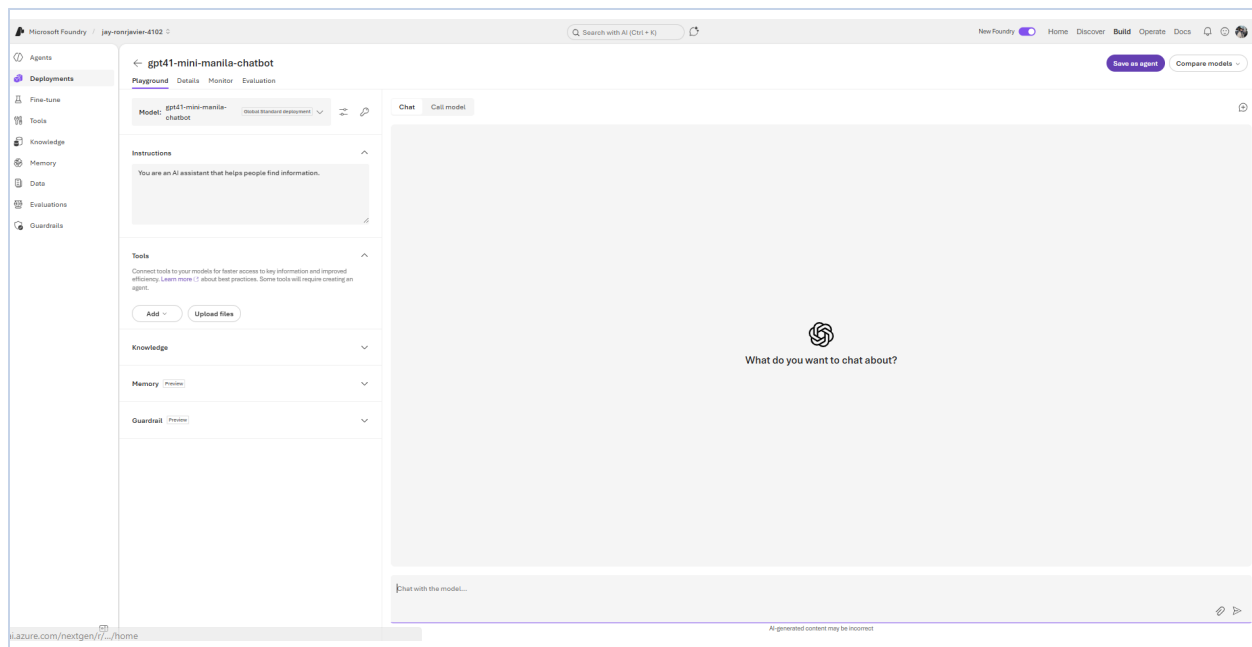
Microsoft Foundry — Create a project: project name jay-ranrjavier-4102, Foundry resource jay-ranrjavier-4102-resource, East US 2, resource group rg-rain-forecast-project-jayron.



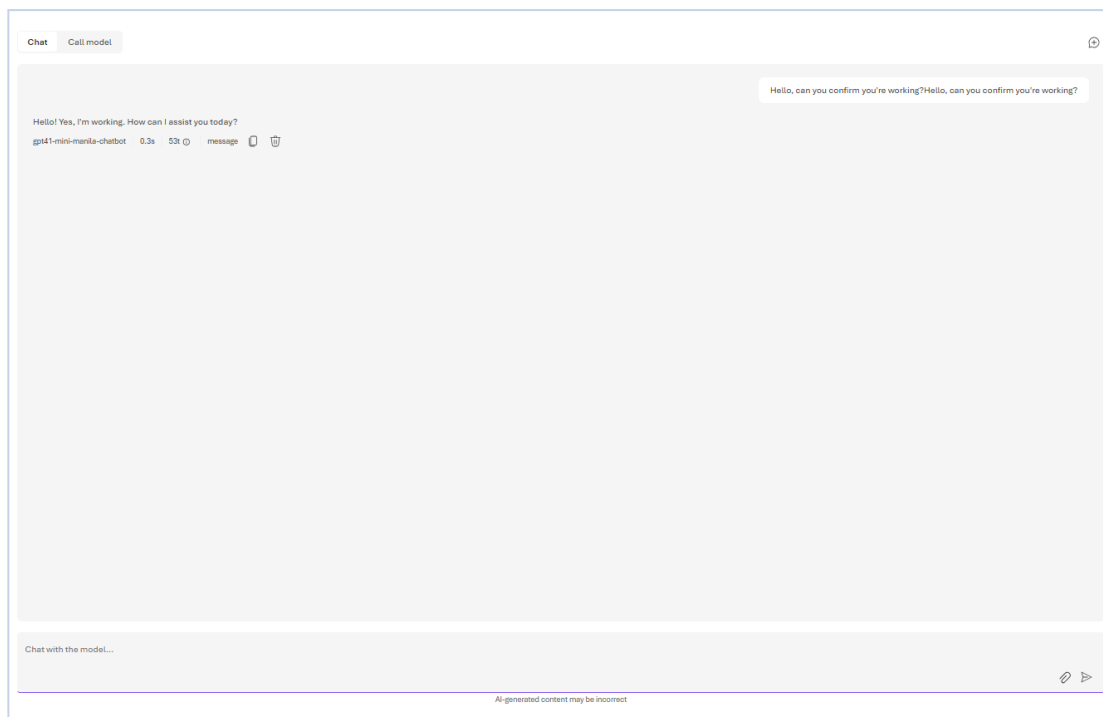
Microsoft Foundry home — jay-ronjavier-4102 project: API key, Project endpoint, and Azure OpenAI endpoint visible.



Microsoft Foundry — Deploy gpt-4.1-mini: deployment name gpt41-mini-manila-chatbot, Pay-per-token, Standard offering, Global scope.



Microsoft Foundry — Deployments: gpt41-mini-manila-chatbot in the Playground, model confirmed active.



Microsoft Foundry — Playground chat test: model responded 'Hello! Yes, I'm working.' confirming deployment is live.

Section 7 · Azure Speech (Text-to-Speech)

Rubric: Azure Cognitive Services Implementation — 20 pts

Every AI Assistant response is automatically read aloud using Azure Speech Service (Jenny Neural voice, en-US). The audio plays immediately after the response appears, with a Replay button for the user to hear it again.

- **Resource** speech-rain-forecast-jayron (East US, Free F0 tier).
- **Implementation** /api/speak route accepts the assistant's reply text, calls the Azure Speech REST API, and returns an audio blob. Auto-plays in the browser via the Audio API.
- **UX Detail** TTS runs in the background — it does not block the chat response from appearing. If autoplay is blocked by the browser, the Replay button still works.

Home > Microsoft Foundry | Speech service

Create Speech Services

Basics Network Identity Tags Review + create

Transcribe audible speech into readable, searchable text. Add real-time speech translations to your apps and services. Convert text to audio nearly in real time. Quickly build speech-enabled apps and services using the programming languages you already work with. Customize speech systems to optimize quality for specific scenarios.

[Learn more](#)

Project Details

Subscription * ⓘ Visual Studio Enterprise Subscription

Resource group * ⓘ rg-rain-forecast-project-jayron [Create new](#)

Instance Details

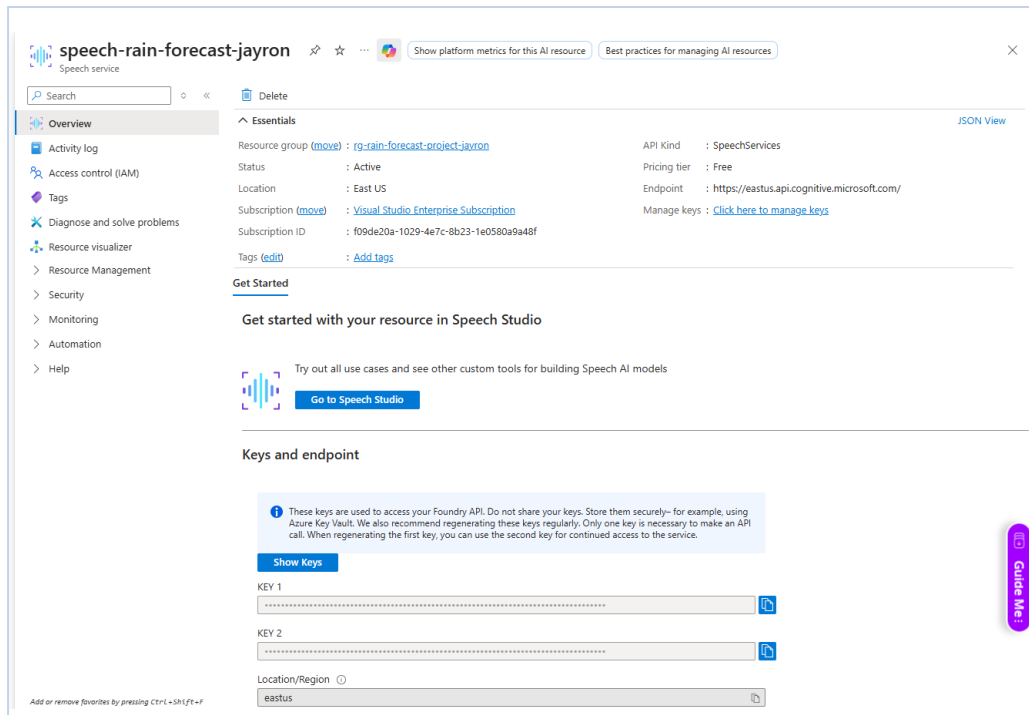
Region ⓘ (US) East US

Name * ⓘ speech-rain-forecast-jayron ✓

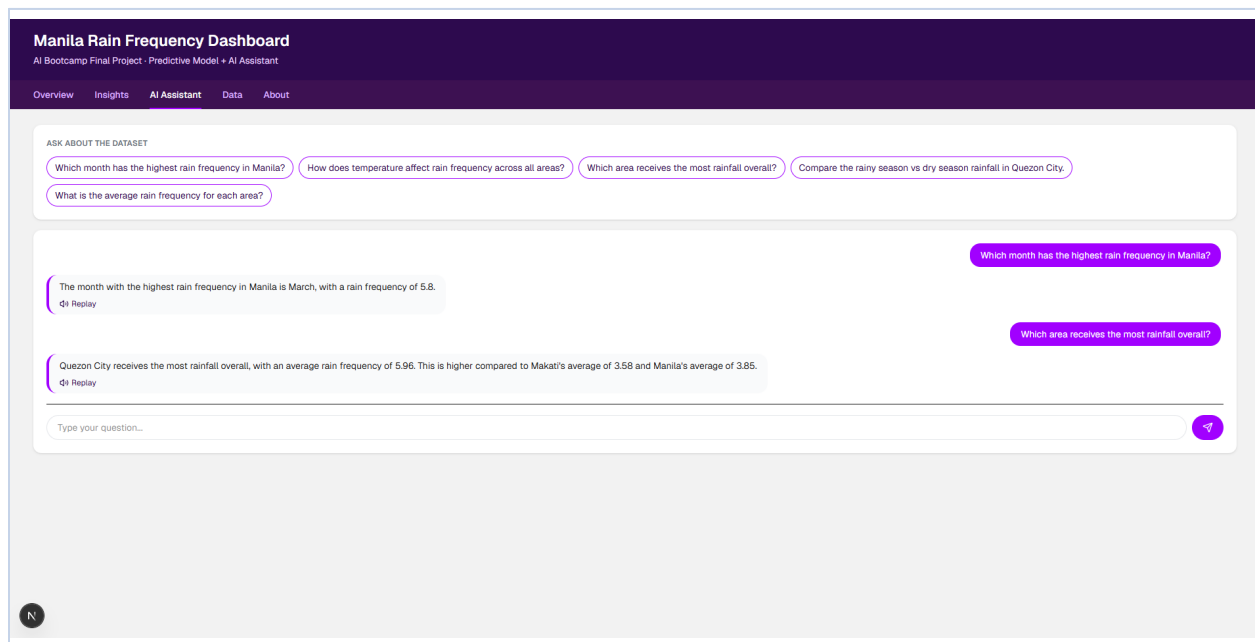
Pricing tier * ⓘ Free F0

[View full pricing details](#)

Azure Portal — Create Speech Services: resource name speech-rain-forecast-jayron, East US, Free F0 tier, resource group rg-rain-forecast-project-jayron.



Azure Portal — *speech-rain-forecast-jayron* Overview: Status Active, East US, Free pricing tier, endpoint <https://eastus.api.cognitive.microsoft.com/>



Web App — AI Assistant tab: predefined questions, two answered questions with Replay buttons, confirming TTS integration is live.

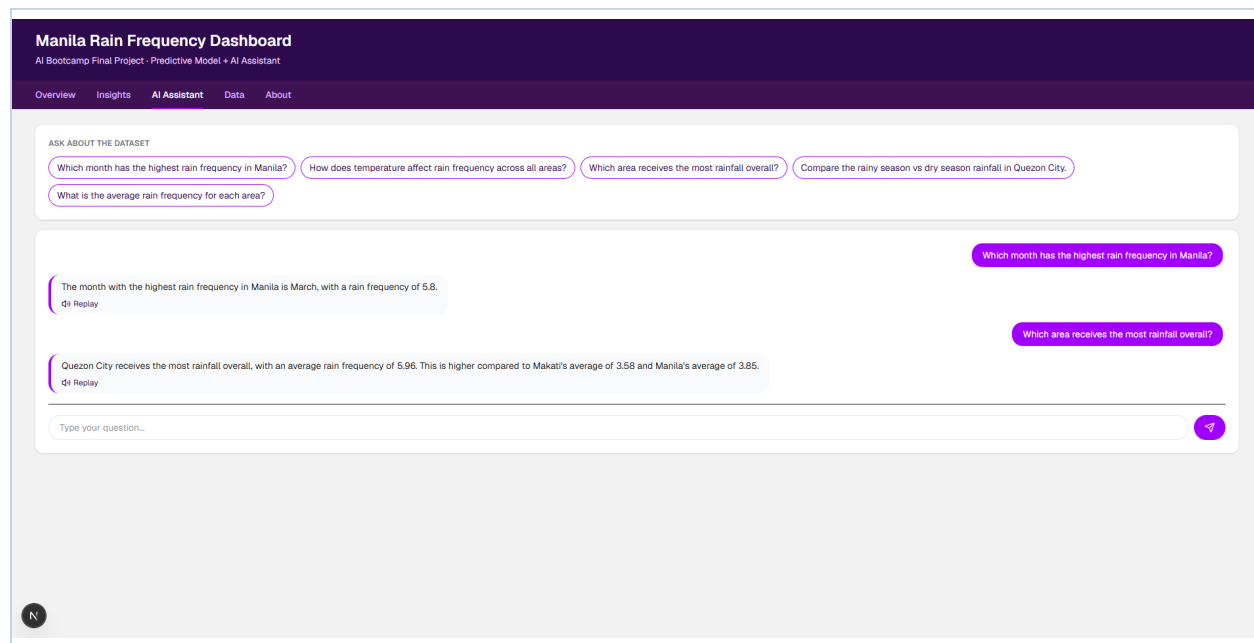
Section 8 · Prompt Engineering

Rubric: Prompt Engineering — 15 pts

The system prompt embeds the full dataset context — summary stats, area breakdowns, model formula, and the cleaned dataset — so the assistant answers specifically about Manila rainfall, not generically.

- Dataset Context** Dataset overview, per-area summary stats, and model RMSE/R²/formula are injected into every system prompt. The full 36-row dataset is included as fallback for specific record queries.
- Temperature** Set to 0.5 — factual and consistent answers, with slight flexibility for explanation phrasing.
- Predefined Questions** 5 questions cover the 2 spec-required queries plus 3 extras. Each is answerable directly from the embedded dataset context without hallucination risk.

Question	Why It's Included
Which month has the highest rain frequency in Manila?	Spec-required — tests dataset query
How does temperature affect rain frequency across all areas?	Spec-required — tests model understanding
Which area receives the most rainfall overall?	Area comparison — shows cross-area analysis
Compare rainy season vs dry season in Quezon City.	Seasonality — demonstrates temporal reasoning
What is the average rain frequency for each area?	Summary stats — tests per-area aggregation



Web App — AI Assistant tab: 5 predefined questions shown, two answered ('Which month has the highest rain frequency in Manila?' and 'Which area receives the most rainfall overall?') with Replay buttons.

Section 9 · Web App — Features & Architecture

Rubric: Create & Integrate AI to Web App — 15 pts

The web app is a Next.js dashboard deployed on Vercel with 5 tabs. All Azure keys are server-side only. The active dataset is stored in Azure Blob Storage and switchable live without redeployment.

- **Overview Tab** KPI cards (total rows, areas covered, average rain frequency, model R^2) plus a line chart of rain frequency by month per area. Powered by the active dataset from Azure Blob Storage.
- **Insights Tab** Scatter plot of Temperature vs. Rain Frequency per area, plus a linear regression predictive model chart with RMSE/ R^2 caption.
- **AI Assistant Tab** Chat interface with 5 predefined questions plus free text input. Responses auto-play via Azure TTS, with a Replay button for each response.
- **Data Tab** Passcode-gated dataset upload. Multi-dataset selector dropdown — switch the active dataset live.
- **About Tab** Project summary, tech stack, and a guide to each tab — replaces the earlier Power BI tab.

API Route	What It Does
/api/chat	Accepts message + dataset + history. Calls Azure OpenAI Responses API. Returns reply text.
/api/speak	Accepts reply text. Calls Azure Speech REST API. Returns audio blob.
/api/dataset	Fetches active dataset blob from Azure Storage. Supports ?blobName= param. Falls back to manifest first entry, then legacy blob.
/api/manifest	Returns list of all uploaded datasets from manifest.json. Returns empty list if none yet.
/api/upload	Passcode-gated. Validates dataset structure. Uploads to a timestamped blob. Updates manifest.json.

Important: Beyond minimum: the multi-dataset selector uses a manifest blob pattern — manifest.json indexes all uploaded datasets, with each dataset living in its own timestamped blob. Switching datasets updates Overview, Insights, and the Assistant globally with no redeployment needed.

Compute > New compute > Simple form: ON

Create new compute

General

Compute name
rain-forecast-cluster

Policy
Unrestricted

Performance

☐ Machine learning

Databricks runtime
17.3 LTS Scala 2.13, Spark 4.0.0

☐ Photon acceleration

Preferred node type
Standard_D4ds_v5 16 GB Memory, 4 Cores

☒ Single node

☒ Terminate after 120 minutes of inactivity

Tags

Key Value

> Automatically added tags

Advanced

Azure Databricks — Create new compute: rain-forecast-cluster, Runtime 17.3 LTS, Standard_D4ds_v5, Single node, terminate after 120 min idle.

Upload files to a Volume in Unity Catalog

Files
Upload any file format up to 5GB to Unity Catalog for analytics, AI and file-centric workflows such as ML on unstructured data, staging raw data before processing, or storing libraries and configuration files.

Rain Forecast Final Project Dataset.csv

☐ Overwrite files with the same filename

Destination volume
Volumes are locations for storing files in Unity Catalog. Select an existing one or create a new Volume.

Search by volume, catalog, or schema name + Create volume

For you Volumes you can upload to All volumes

You do not have any recents or favorites yet
No favorites or recent volumes

View volumes you can upload to

Volumes/Catalog/Schema/ No volume selected

Cancel Upload

Azure Databricks — Upload files to Unity Catalog: Rain Forecast Final Project Dataset.csv selected, destination volume selector.

Create a new volume

Volumes are locations for storing files in Unity Catalog.

Volume name*

Volume type [Learn more](#)

☒ Managed volume
Files are stored in a location managed by Unity Catalog.

☐ External volume
Files are stored in an external location that you configure.

Choose catalog and schema*

Volumes are organized under a catalog and a schema. Select existing or create new.

[View volumes you can upload to](#)

Azure Databricks — Create a new volume: `rain_forecast_data`, Managed volume, catalog `databricks_rain_forecast_jayron`, schema `default`.

Catalog

Serverless Starter Warehouse Serverless

Type to search...

For you / All

My organization

databricks_rain_forecast_jayron

default

rain_forecast_data

information_schema

system

adb_test_mcd

autorent

autorent_catalog

ch_poc

micatalog

samplicatalog

seancatalog

smo_catalog

w_databricks_sascond

workshopautorent

Shims Received

samples

Legacy

hive_metastore

Catalog Explorer > databricks_rain_forecast_jayron > default >

rain_forecast_data

Overview Files Details Permissions

Description

AI generate Add

/Volumes/databricks_rain_forecast_jayron/default/rain_forecast_data

Filter files and directories at this level

Name	Size	Last modified
Rain Forecast Final Project Dataset.csv	1.25 KB	13 minutes ago
		49 seconds ago

dataset_context.json

Copy path

Download

```
{
  "dataset_overview": [ - ], // 4 items
  "summary_per_area": [ - ], // 3 items
  "model_info": [ - ], // 5 items
  "full_cleaned_dataset": [ - ], // 38 items
}
```

Close

About this volume

Owner jay-ron.javier@accenture.com

Tags

Add tags

Share

Upload to this volume

Azure Databricks — Unity Catalog volume `rain_forecast_data` showing both CSV and `dataset_context.json` files uploaded.

```
<> + Code + Text + Genie Code
Python 7
import json

# Summary stats per area
summary_per_area = df.groupby('Area').agg(
    avg_temperature=('Temperature', 'mean'),
    avg_rain_frequency=('Rain Frequency', 'mean'),
    min_temperature=('Temperature', 'min'),
    max_temperature=('Temperature', 'max'),
    min_rain_frequency=('Rain Frequency', 'min'),
    max_rain_frequency=('Rain Frequency', 'max')
).round(2).reset_index()

overall_correlation = df['Temperature'].corr(df['Rain Frequency'])

dataset_summary = {
    "dataset_overview": {
        "total_rows": len(df),
        "areas": df['Area'].unique().tolist(),
        "months_covered": df['Month'].unique().tolist(),
        "temperature_rain_correlation": round(overall_correlation, 4)
    },
    "summary_per_area": summary_per_area.to_dict(orient='records'),
    "model_info": {
        "model_type": "Linear Regression (Temperature -> Rain Frequency)",
        "r_sq": round(rsq, 4),
        "r1_score": round(r2, 4),
        "intercept": round(model.intercept_, 4),
        "coefficient_temperature": round(model.coef_[0], 4)
    },
    "full_cleaned_dataset": df.to_dict(orient='records')
}

# Write to Volume so you can download it from Databricks UI
output_path = '/Volumes/databricks_rain_forecast_jayron/default/rain_forecast_data/dataset_context.json'
with open(output_path, 'w') as f:
    json.dump(dataset_summary, f, indent=2)

print(f"JSON exported to: {output_path}")
print("\n=== PREVIEW ===")
print(json.dumps(dataset_summary, indent=2)[:1500])

> summary_per_area: pandas.core.frame.DataFrame = [Area: object avg_temperature: float64 ... 5 more fields]
JSON exported to: /Volumes/databricks_rain_forecast_jayron/default/rain_forecast_data/dataset_context.json
=== PREVIEW ===
```

Databricks Notebook — Cell 7 code: building `dataset_summary` JSON with overview, per-area stats, model info, full cleaned dataset, then writing to Volume path.

```
print(json.dumps(dataset_summary, indent=2)[:1500])

> summary_per_area: pandas.core.frame.DataFrame = [Area: object avg_temperature: float64 ... 5 more fields]
{
  "areas": [
    "Makati",
    "Quezon City",
    "Manila"
  ],
  "months_covered": [
    "January",
    "February",
    "March",
    "April",
    "May",
    "June",
    "July",
    "August",
    "September",
    "October",
    "November",
    "December"
  ],
  "temperature_rain_correlation": 0.325
},
  "summary_per_area": [
    {
      "Area": "Makati",
      "avg_temperature": 23.42,
      "avg_rain_frequency": 3.58,
      "min_temperature": 18.0,
      "max_temperature": 28.0,
      "min_rain_frequency": 2.1,
      "max_rain_frequency": 5.5
    },
    {
      "Area": "Manila",
      "avg_temperature": 27.25,
      "avg_rain_frequency": 3.85,
      "min_temperature": 22.0,
      "max_temperature": 32.0,
      "min_rain_frequency": 2.6,
      "max_rain_frequency": 5.8
    },
    {
      "Area": "Quezon City",
      "avg_temperature": 25.45,
      "avg_rain_frequency": 5.96,
```

Databricks Notebook — Cell 7 output: `dataset_context.json` exported, JSON preview showing areas, months_covered, temperature_rain_correlation, and summary_per_area.

Microsoft Azure

Home > Storage center | Blob Storage

Create a storage account

Basics Advanced Networking Data protection Security Encryption Tags **Review + create**

[View automation template](#)

Basics

Subscription	Visual Studio Enterprise Subscription
Resource group	rg-rain-forecast-project-jayron
Location	East US
Storage account name	strainforecastjayron
Primary service	
Performance	Standard
Replication	Locally redundant storage (LRS)

Advanced

Enable hierarchical namespace	Disabled
Enable SFTP	Disabled
Enable network file system v3	Disabled
Allow cross-tenant replication	Disabled
Access tier	Hot
Managed Identity for SMB	Disabled

Azure Files

Require Encryption in Transit for SMB	Enabled
---------------------------------------	---------

Security

Secure transfer	Enabled
Blob anonymous access	Disabled
Allow storage account key access	Enabled

[Previous](#) [Next](#) [Create](#)

Azure Portal — Create a storage account: strainforecastjayron, East US, Standard LRS, resource group rg-rain-forecast-project-jayron.

9888 | Overview > strainforecastjayron

Containers

+ Add container Upload Refresh Delete Change access level Restore containers Edit columns

Search containers by prefix

Showing all 1 items

☐	Name	Last modified	Anonymous access level
☐	Slogos	6/18/2026, 11:52:10 PM	Private

New container

Name * rainforecast

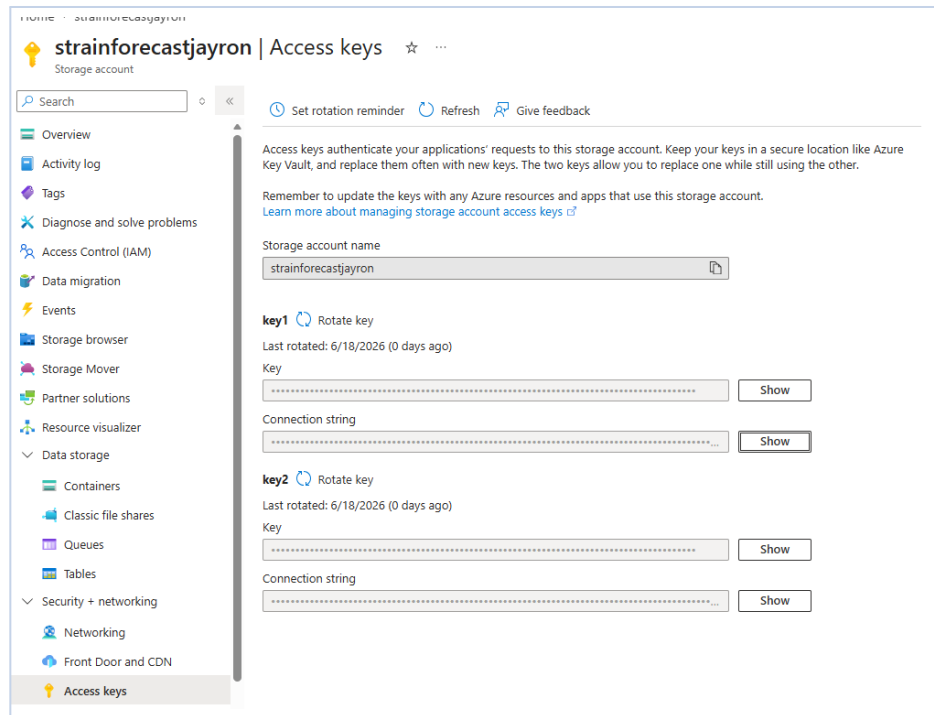
Anonymous access level Private (no anonymous access)

The access level is set to private because anonymous access is disabled on this storage account.

Advanced

Guide Me

Azure Portal — New container creation: name rainforecast, Private access level.



Azure Portal — strainforecastjayron Access keys: key1 and key2 with connection strings for use in Vercel environment variables.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\jay-ron.r.javier\Desktop\manila-rain-chatbot>npm install @azure/storage-blob

added 26 packages, and audited 419 packages in 13s

155 packages are looking for funding
  run `npm fund` for details

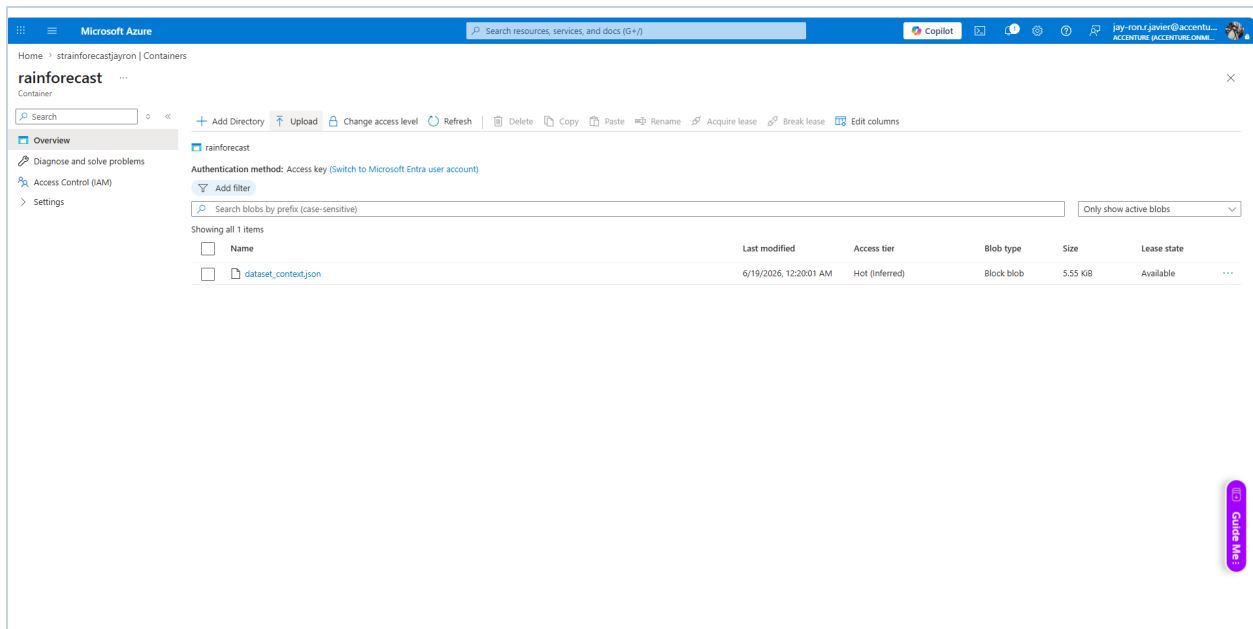
2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

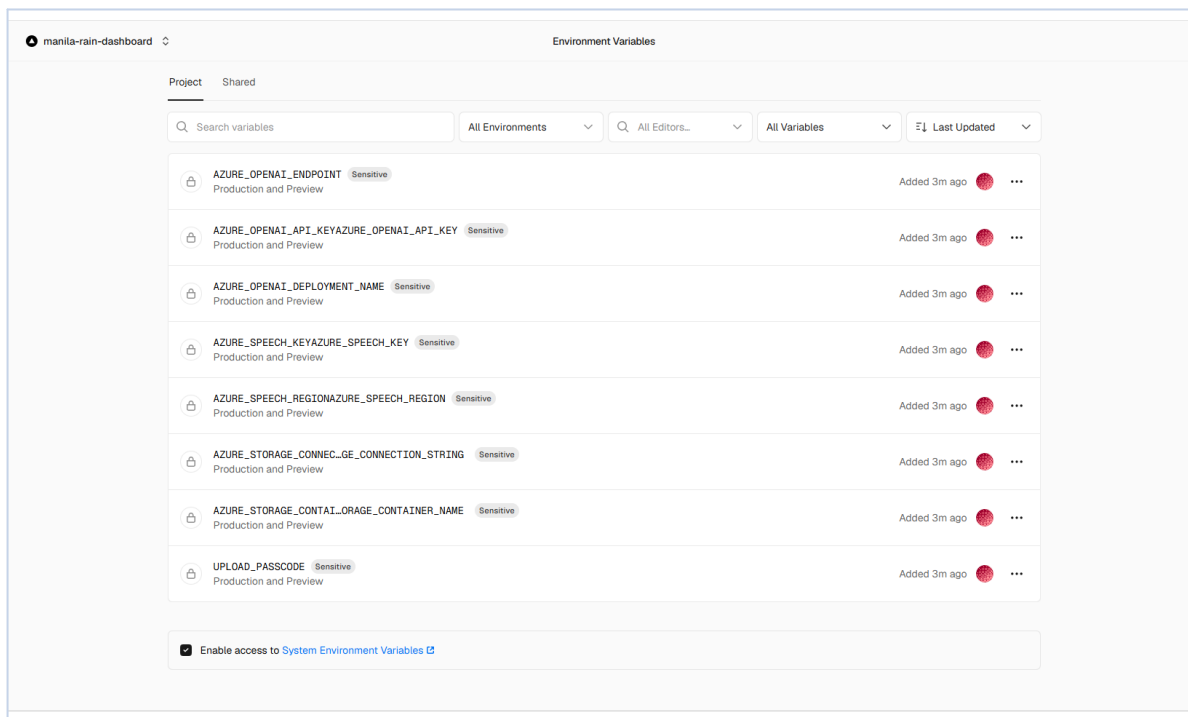
Run `npm audit` for details.

C:\Users\jay-ron.r.javier\Desktop\manila-rain-chatbot>
```

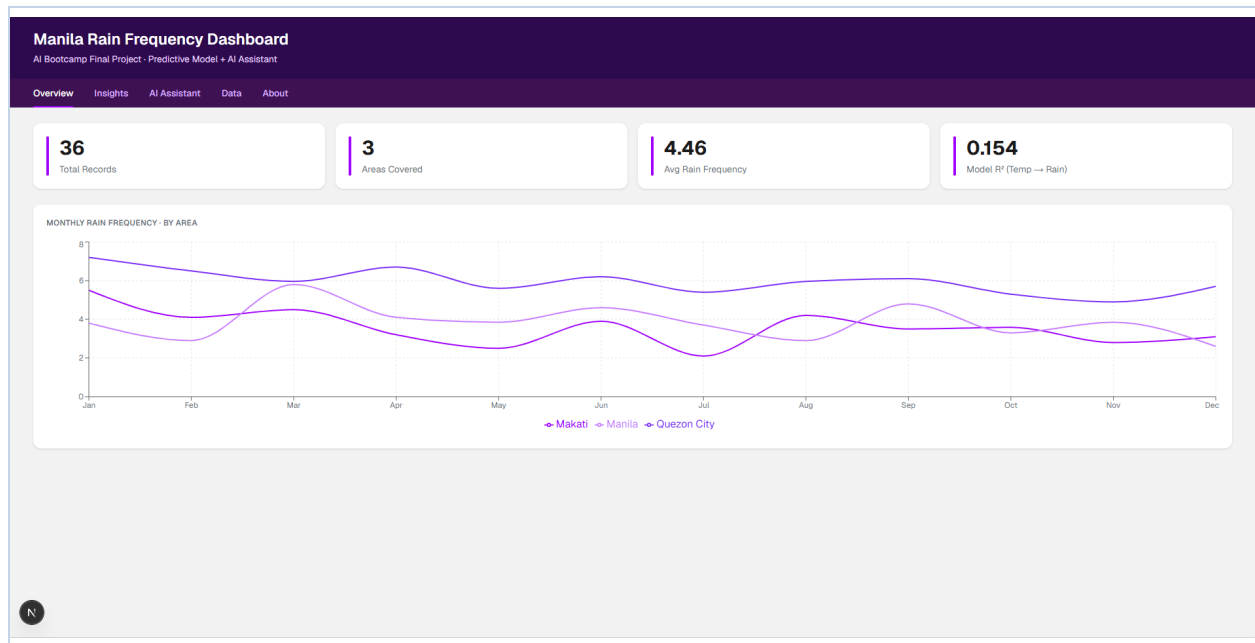
VS Code Terminal — npm install @azure/storage-blob completed successfully in manila-rain-chatbot project.



Azure Portal — *rainforecast* container: *dataset_context.json* blob uploaded, 5.55 KiB, Block blob, Available.



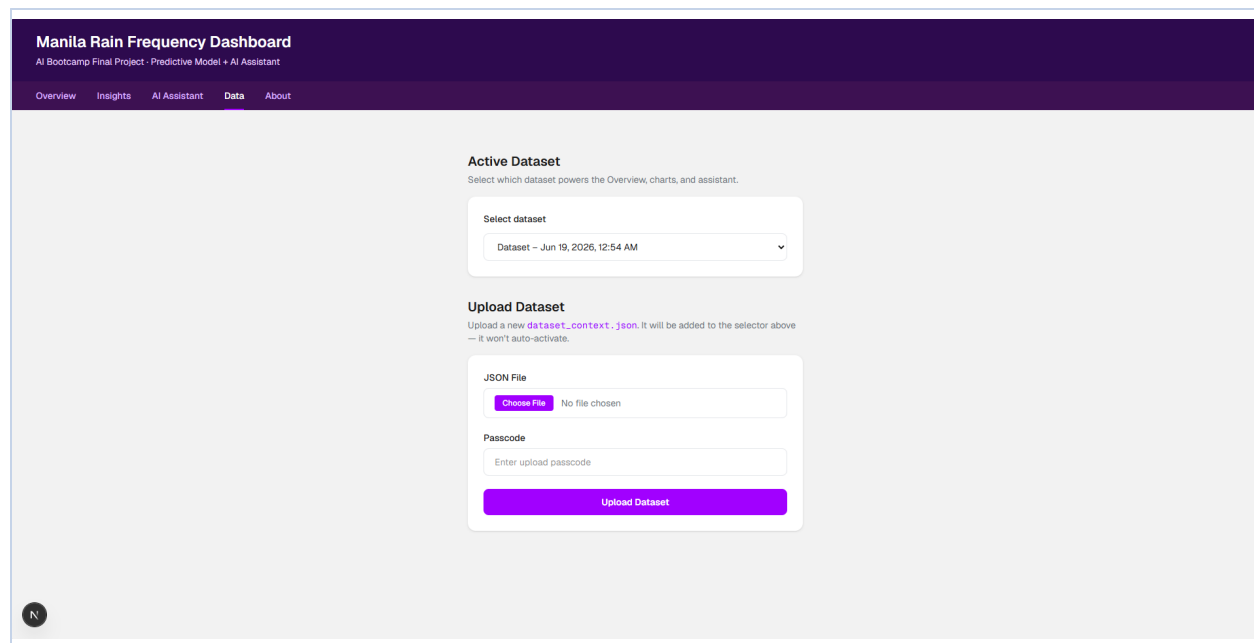
Vercel — *manila-rain-dashboard* Environment Variables: *AZURE_OPENAI_ENDPOINT*, API keys, *AZURE_SPEECH* keys, *AZURE_STORAGE* connection string, container name, and *UPLOAD_PASSCODE* all set as Sensitive.



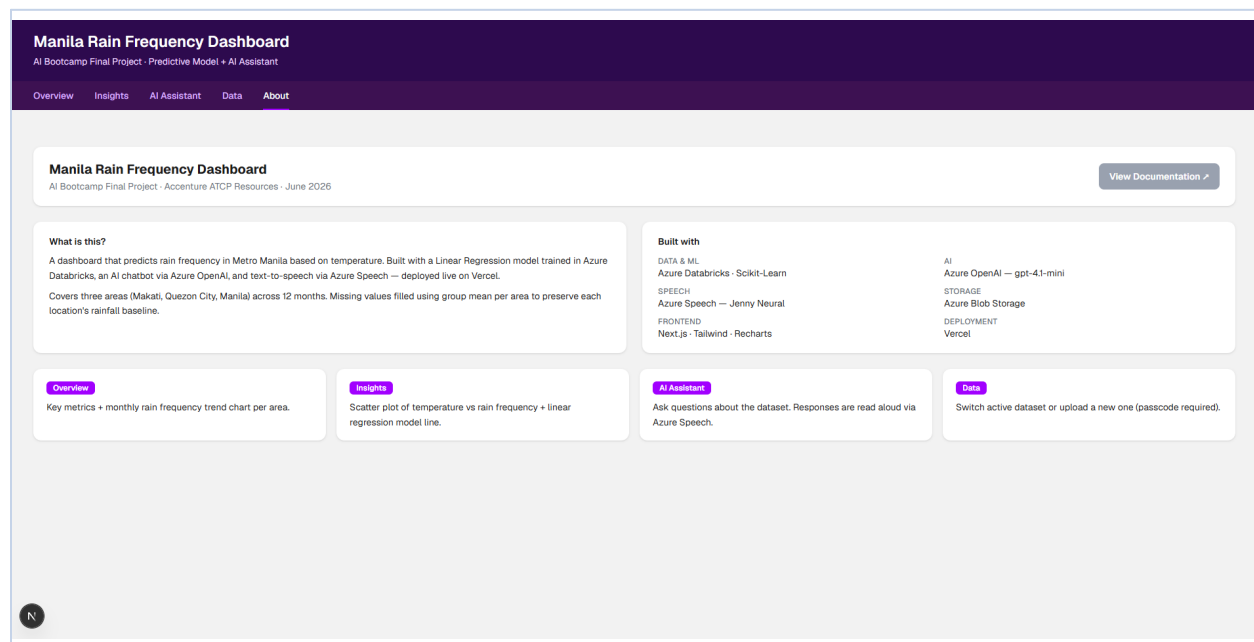
Web App — Overview tab: 36 total records, 3 areas, avg rain frequency 4.46, Model R^2 0.154, monthly rain frequency line chart by area.



Web App — Insights tab: Temperature vs. Rain Frequency scatter plot (top) and Predictive Model linear regression chart (bottom).



Web App — Data tab: Active Dataset selector dropdown (Dataset – Jun 19 2026, 12:54 AM) and Upload Dataset section with passcode field.



Web App — About tab: project summary, tech stack (Azure Databricks, OpenAI gpt-4.1-mini, Azure Speech, Next.js, Tailwind, Recharts, Vercel), and tab descriptions.

Section 10 · Q&A Preparation

Rubric: Presentation — 20 pts

Prepared answers for likely questions from the panel.

Question	Answer
Why Linear Regression?	Small dataset (36 rows). Linear Regression is interpretable and appropriate for this size. Random Forest or XGBoost would overfit.
Why is R^2 only 0.154?	Temperature alone explains roughly 15% of Rain Frequency variation with 36 rows. Adding Area raises R^2 to 0.634 — location is the stronger driver. Model 1 is kept as primary since it matches the spec.
How does prompt engineering work?	The system prompt embeds the full dataset summary and model formula so the assistant answers specifically about Manila. Temperature 0.5 keeps answers factual.
Why Next.js instead of Flask?	More productive in Next.js. API routes provide a secure server-side layer for Azure keys. Vercel makes deployment trivial.
Is the app secure?	Azure keys are stored as Vercel environment variables. Only accessed in server-side API routes. Never exposed to client-side code or committed to GitHub.
What would you improve?	Add speech-to-text so users can ask by voice. Add more historical data for a better model. Add area filtering so users can compare areas side by side.
What is the TTS for?	Azure Speech reads every chatbot response aloud — integrating Azure Cognitive Services meaningfully, not just as a checkbox feature.
What challenges did you face?	gpt-35-turbo was deprecated; the Foundry-managed resource had a different endpoint than expected; quota limits on Global deployment scope; the missing-value strategy required a group mean, not a global mean.